Envio de documentos pelo grupo do Telegram · status de entrega

> **Sem número de bloco.** Trilha da automação, lista **"OCR Contratos (WhatsApp) — Breno"** do ClickUp. Ver `docs/README.md` e o `CLAUDE.md`.

Data: 2026-10-27 · Ambiente: gc-dev (`gzbmhgnpoehormnidmgg`) · Plano: `docs/automacao/migracao-telegram-integracao-automacao.md` , decisões 33 a 38 (descritas abaixo).

Resumo

Implementado e validado em gc-dev: as sete camadas do `npm run validar` passaram nesta rodada. Falta o que só o Breno faz: o teste real num grupo do Telegram (Fase 0), o merge e deploy, e a publicação do workflow do n8n (seções 5 e 6).

Pessoas liberadas pelo admin em **Configurações → Bot → Usuários permitidos** passam a mandar PDF de proposta ou contrato ao bot **também num grupo liberado**, chamando o bot. Quem não está liberado e tenta vira um **pedido de acesso** que o admin aprova com um clique. A regra de quem pode mora numa função pura (`src/lib/bot-acesso.ts`), executada por uma rota nova do sistema (`POST /api/ingestao/triagem-telegram`) que o n8n chama.

Energia: a checagem de bateria marcou 43% (acima do limite de 30%). A validação rodou sob `caffeinate -i` e não dormiu. Nenhum risco de energia a registrar.

1. Decisões tomadas pelos agentes (a gestão não responde)

Cada uma é a opção mais segura e simples.

#	DECISÃO	MOTIVO
33	**Um grupo da equipe, não um grupo por obra.** Grupo liberado não tem obra; a obra continua saindo do PDF (decisão 27).	Um grupo por obra criaria segunda fonte de obra que pode contradizer o PDF. A tabela aceita vários grupos por empresa, então mudar depois não custa.
34	**Liberar por pessoa, não o grupo todo.** Só vale se grupo **e** pessoa estão liberados.	Quem controla os membros do grupo é quem o administra, não o admin do sistema.
35	**Bot fora da administração do grupo, privacy ligado, envio chamando o bot**: legenda `/enviar` , menção ao bot ou PDF em resposta a mensagem do bot. O resto do grupo é ignorado sem consultar o banco.	Protege o limite de execuções do n8n (decisão 22) e funciona também se o bot virar admin.

#	DECISÃO	MOTIVO
36	**Quem é liberado vale no grupo e na conversa privada.**	Na conversa privada `chat.id` é o id da pessoa; uma permissão, um lugar para revogar. Quem já era cadastrado vale no grupo sem migrar dado.
37	**Técnica: a triagem roda no sistema**, não no n8n.	Padrão desde a decisão 26: a regra tem uma cópia só, coberta pelo plano de validação.
38	**Técnica: o workflow do n8n não foi alterado.**	O workflow `thlKSZcBPX84Y51N` é o de produção e chamaria uma rota que só existe depois do deploy; publicar antes derrubaria o bot. O código novo é compatível com o n8n de hoje.

Decisões menores da revisão: mensagem de grupo sem remetente (`from`) conta como anônima (senão o id do grupo viraria remetente e violaria o CHECK, erro 500); `/configuracoes/contatos` redireciona para `/configuracoes/bot` depois do guard de admin; a tabela de pedidos se chama `bot_pedidos_acesso` (já existe `pedidos_acesso` , do "esqueci minha senha").

2. Como ficou

- **Banco** (`supabase/migrations/20261027100000_bot_envio_pelo_grupo.sql`): `contatos_whatsapp.ativo` (revogar sem apagar); tabelas `bot_grupos` (índice único no `telegram_chat_id`) e `bot_pedidos_acesso` (um por pessoa por empresa); `documentos_processamento` ganha remetente e id da mensagem; triggers de auditoria e `updated_at` ; RLS só com leitura e mudança de status pelo admin, **sem policy de insert nem de delete**.
- **Regra pura** `src/lib/bot-acesso.ts` : `triarMensagemTelegram` decide entre processar, pedir acesso, avisar ou ignorar. Tabela de casos G1 a G21 e P1 a P7 testada.
- **Rota** `POST /api/ingestao/triagem-telegram` (token de ingestão): só consulta o banco depois de ler a identidade; grupo de admin é ignorado antes de qualquer consulta; grupo novo entra PENDENTE.
- **`/api/ingestao/resposta-obra`** : no grupo, filtra também pelo remetente; sem `remetenteId` se comporta como hoje.
- **Telas** em Configurações → Bot: Usuários permitidos (substitui a antiga aba Contatos, que redireciona), Pedidos de acesso (com selo de pendentes) e Grupos (Liberar e Bloquear com confirmação). Oito Server Actions, cada uma repete a checagem de admin e filtra por `empresa_id` .
- **Auditoria**: as duas tabelas entram em `ENTIDADE_LABELS` (de 15 para 17 entidades).

3. Arquivos

ARQUIVO	O QUÊ
`supabase/migrations/20261027100000_bot_envio_pelo_grupo.sql`	migration (aplicada em gc-dev)
`src/lib/bot-acesso.ts` + `.test.ts`	regra pura da triagem
`src/lib/bot-config.ts` + `.test.ts`	helpers de tela
`src/lib/contatos.ts`, `telegram.ts`, `auditoria.ts` (+ testes)	ajustes
`src/lib/supabase/types.ts`	tipos regerados (+98 linhas)
`src/app/api/ingestao/triagem-telegram/route.ts`	rota nova
`src/app/api/ingestao/resposta-obra/route.ts`	filtro por remetente no grupo
`src/app/(app)/configuracoes/bot/`	`layout`, `page`, `abas`, `actions`, `usuarios-permitidos-panel`, `pedidos/`, `grupos/`
`src/app/(app)/configuracoes/{abas,layout}.tsx`, `contatos/page.tsx`	aba Bot e redirect (`contatos/actions.ts` e `contatos-panel.tsx` saíram)
`src/app/(app)/documentos/actions.ts`	ajuste
`scripts/validacao/*`	plano estendido (seção 4)

4. Validação

`caffeinate -i npm run validar` em gc-dev, 2026-10-27: **todas as camadas passaram**. Antes, `bash scripts/validacao/validar.sh build --aceitar-rotas` regravou `rotas-esperadas.txt` (66 para 70 rotas; as 4 novas: `/api/ingestao/triagem-telegram`, `/configuracoes/bot`, `/configuracoes/bot/grupos`, `/configuracoes/bot/pedidos`).

CAMADA	RESULTADO (LIDO DO OUTPUT DESTA RODADA)
estático (`tsc` + `next lint`)	limpo; "No ESLint warnings or errors" (11 s)
unitário (`npm test`)	**480 casos**, 480 passam, 0 falham (6 s)
build (`next build` + rotas)	**70 rotas**, iguais à baseline (45 s)
runtime (`next start` + fetch)	**334/334 rotas ok** (273 s)
dados (RLS em gc-dev)	**68/68 checagens ok**, 4 sem registro em gc-dev (79 s)

CAMADA	RESULTADO (LIDO DO OUTPUT DESTA RODADA)
escrita (Server Actions por HTTP)	**575/575 passos ok**, mais **17/17 passos** do ensaio da importação (420 s)
navegador (Chrome headless sobre `next dev`)	**286/286 passos ok**, screenshots em `/tmp/gc-validacao/shots` (553 s)

O que o plano ganhou (obrigação 1)

SCRIPT	ACRÉSCIMO
`validacao-rotas.json`	as rotas do Bot por perfil (admin abre; comercial e visualizador recusados)
`rotas-esperadas.txt`	as 4 rotas novas
`matriz-rls.json`	`bot_grupos` e `bot_pedidos_acesso` (select e update só do admin, insert e delete vazios) e as rotas novas
`validar-dados.mjs`	3 consultas das páginas copiadas dos `page.tsx`; a matriz cria uma linha `VALIDA-MATRIZ-` em cada tabela nova e apaga no `finally`
`validar-escrita.mjs`	20 checagens: triagem (401, privado de desconhecido, sem chamada, grupo de admin, grupo novo PENDENTE, grupo liberado, resposta, pedido, anônimo, grupo conhecido, contato inativo, migração de grupo) e resposta-obra no grupo
`validar-navegador.mjs`	aba Bot, desativar contato, selo de pendentes, Permitir, Liberar e Bloquear, redirect de `/contatos`
`verificar-sobras.mjs`	procura `bot_pedidos_acesso`, `bot_grupos` e `contatos_whatsapp` por `VALIDA-%`

5. O que não foi validado — pendências nominais

1. **Fase 0, teste real num grupo do Telegram com o celular do Breno**, com o bot no grupo, privacy **ligado** e o workflow publicado. Anotar se chega ao n8n: (a) PDF com legenda `/enviar@<bot>`; (b) PDF com legenda de menção ao bot; (c) PDF em resposta a mensagem do bot; (d) PDF solto (deve ser ignorado). Repetir com o bot como admin do grupo. Se (a) e (b) não chegarem com o privacy ligado, a instrução às pessoas vira "responda a uma mensagem do bot com o PDF"; o código não muda.
2. **Resposta em citação num grupo real e migração real para supergrupo**: só exercitadas por HTTP contra a rota, com mensagens fabricadas.
3. **Workflow do n8n não foi publicado** (decisão 38); o workflow de 40 nós está pronto em `scripts/automacao/n8n-processar-documento-grupo.json` (ver a seção 5.1). Publicar é do Breno,

depois do deploy. **Não foi exercitado no n8n**: a validação é estática (estrutura, conexões e expressões), e o teste real é a Fase 0.

4. **Consultas "não exercitadas" na camada dados** (0 registro em gc-dev, query válida): `bot_pedidos_acesso` (listagem), `bot_grupos` (listagem) e a contagem de pedidos pendentes. A matriz de RLS cria linhas de teste, mas as consultas das páginas não. O quarto "sem registro" é a de aditivos de contrato.
5. (Resolvida) As decisões 33 a 38 foram copiadas para `migracao-telegram-integracao-automacao.md`.
6. **Detalhe de UX mantido como a spec**: no grupo PENDENTE, contato ativo de outra empresa também recebe a mensagem de grupo pendente.
7. **Conferência visual** dos screenshots e **teste no celular** (padrão da sprint 8): manuais, não feitos.
8. A variável `TELEGRAM_BOT_USERNAME` ainda não existe na Vercel; sem ela a menção ao bot não é reconhecida em produção.

5.1 Workflow do n8n (arquivo, não publicado)

- **Arquivo:** `scripts/automacao/n8n-processar-documento-grupo.json` (40 nós: 38 do ativo, menos `Supabase - Buscar contato`, mais `Rota - Triagem do Telegram`, `Responder em citação?` e `Telegram - Responder`). Passo a passo e como voltar: `scripts/automacao/README.md`.
- **Base:** cópia fiel do workflow ativo `thlKSZcBPX84Y51N` (versionId `3598b2b4-a6fd-47d0-bdf7-eabb909cb686`, lido em 27/10 só por leitura; nada foi escrito no n8n). Backup em `/Users/a1234/.claude/jobs/c6e80632/tmp/n8n-backup-thlKSZcBPX84Y51N.json` (fora do repositório).
- **`validate_workflow`** (perfil runtime): **0 erros, 0 avisos**, 54 conexões válidas, 0 inválidas, 32 expressões validadas, 39 nós habilitados (a nota adesiva não conta). 5 sugestões, nenhuma exigida (alwaysOutputData em Gemini, Groq e Rota - Gravar no sistema; tratamento de erro; dividir o workflow), todas sobre nós que não foram tocados. **Limite honesto:** a validação rodou numa cópia em que o código dos Code nodes **não alterados** foi trocado por um stub (o envio inteiro passava de 50 KB); os 13 Code nodes do arquivo final, com o código completo, tiveram a **sintaxe conferida** por parse local do JavaScript, sem erro.
- **Nó Telegram:** `get_node` confirma `additionalFields.reply_to_message_id` (número). O `Telegram - Responder` usa `={{ $json.reply_to }}` e `appendAttribution: false`, como o `Telegram - Enviar`. Sem `reply_to` o item segue para o `Telegram - Enviar`, que não mudou.
- **Conversa privada segue o caminho de hoje:** o nó `Telegram Trigger` é o mesmo; a triagem pela rota devolve itens no mesmo formato (`processar`, `resposta`, `responder`) das saídas de `Triar mensagem` de antes (os textos da privada vêm de `TEXTOS_BOT`, que pode diferir em palavras), e os nós de `Processar?` em diante só ganharam campos novos (`remetente_id`, `mensagem_id`, `reply_to`), nulos ou ignorados fora do grupo. Texto da privada de "sem pergunta aberta" é o de sempre.
- **Não exercitado:** nenhuma execução do workflow novo, nem em teste. O que foi conferido é o contrato de entrada e saída da rota (`route.ts` e `itensDaAcao`) contra o código dos nós.
- **Se a rota cair:** `Triar mensagem` responde na privada "Tivemos um problema ao receber sua mensagem" e avisa o grupo de admin (no grupo, só avisa o admin quando a mensagem tem documento).

6. Registro para quem for operar gc-prod

gc-prod está fora do escopo e INACTIVE; nada foi aplicado lá.

- **Migration pendente:** `supabase/migrations/20261027100000_bot_envio_pelo_grupo.sql` (aditiva: coluna `ativo`, duas tabelas, colunas em `documentos_processamento`, triggers e RLS; o rollback está comentado no fim do arquivo). Aplicar depois das anteriores, com backup.
- **Ordem de entrada em produção:** migration, merge e deploy na Vercel, variáveis `TELEGRAM_BOT_USERNAME` e `TELEGRAM_BOT_TOKEN` na Vercel, publicar o workflow do n8n, teste real (Fase 0).
- O deploy pode ir antes do n8n: sem `remetenteId` a `resposta-obra` funciona como hoje.
- Avisar a equipe como mandar: `/enviar@<bot>` na legenda do PDF.